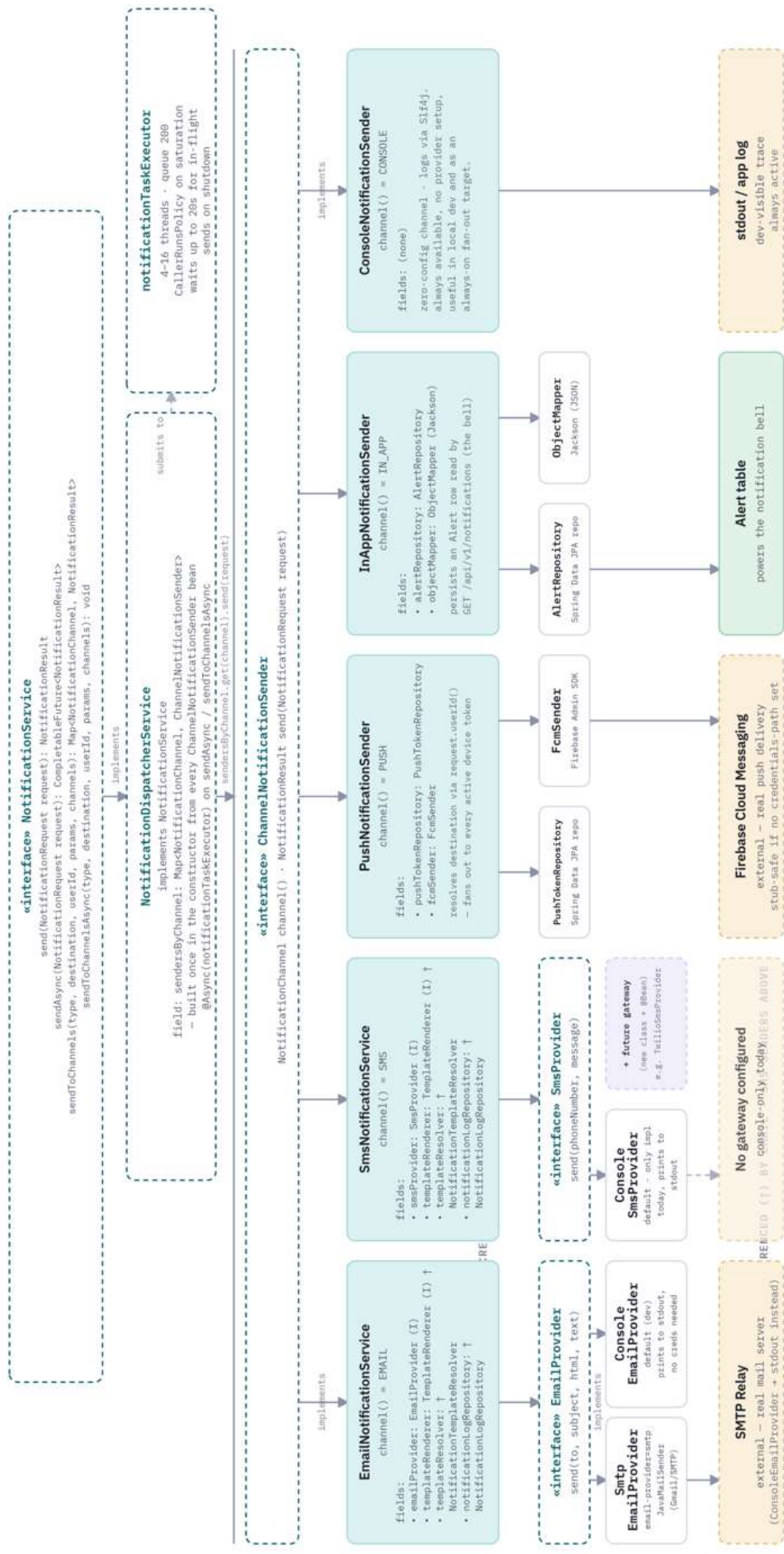


Scalable Notification System



Step 1 — Notification Domain Model

```
class Notification {  
    String id;  
    String userId;  
    String eventType;  
    String templateId;  
    NotificationPriority priority;  
    NotificationStatus status;  
    Map<String, Object> data;  
    Instant scheduledAt;  
    String idempotencyKey;  
}
```

```
class NotificationDelivery {  
    String id;  
    String notificationId;  
    String channel;  
    String provider;  
    String status;  
    Int attemptCount;  
  
    String providerId  
    String errorCode;  
    String errMessage;  
    Timestamp sentAt;  
    Timestamp deliveredAt;  
}
```

```
class NotificationPreference {  
    String id;  
    String userId;  
    String eventType;  
    String channel;  
    boolean enabled;  
}  
UNIQUE(user_id, event_type,  
channel)
```

```
class Template{  
    String id;  
    String templateCode  
    String eventType;  
    String channel;  
    String locale  
  
    String subject;  
    String body  
    int version;  
    String status  
}
```

Step 2 — Notification Channel

(EMAIL, SMS, PUSH, WHATSAPP, IN_APP, CONSOLE, WEB_SOCKET)

```
public interface NotificationChannel {  
    ChannelType getType();  
    DeliveryResult send(NotificationMessage message );  
}
```

Channel Implementations:

```
Class EmailChannel implements NotificationChannel {  
    private final EmailProvider provider;  
  
    @Override  
    public ChannelType getType() { return ChannelType.EMAIL; }  
  
    @Override  
    public DeliveryResult send(NotificationMessage message) { return provider.send(message); }  
}
```

Step 3 — Provider Abstraction

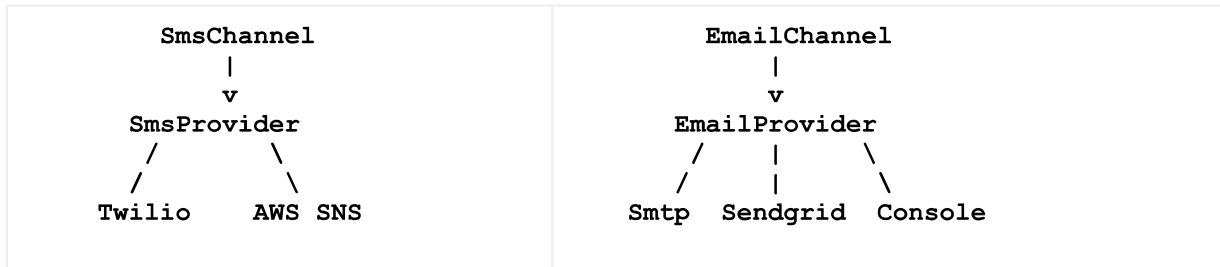
Do not couple `SmsChannel` directly to Twilio.

```
public interface SmsProvider {  
    ProviderResponse send(SmsMessage message);  
    boolean isAvailable();  
}
```

Implementations:

TwilioSmsProvider
AwsSnsSmsProvider
VonageSmsProvider

Architecture:



Step 4 — Provider Factory / Registry

Avoid:

```
if (provider.equals("TWILIO")) {  
    ...  
} else if (provider.equals("AWS")) {  
    ...  
}
```

Instead:

```
public interface ProviderRegistry {  
    <T> T getProvider(  
        ChannelType channel,  
        String providerName  
    );  
}
```

Step 5 — Notification Orchestrator

```
public interface NotificationOrchestrator {  
    NotificationResult process(NotificationRequest request);  
}
```

Notification Request → Validate → Load User Preferences →
Resolve Channels → Resolve Template → Create Notification → Publish Events

Step 6 — Notification Preferences

Users should control notification preferences.

Example: User: U123 ORDER_CONFIRMED PUSH = ON EMAIL = ON SMS = OFF PROMOTIONAL PUSH = OFF EMAIL = OFF SMS = OFF	Database: user_notification_preferences id user_id event_type channel enabled quiet_hours_start quiet_hours_end updated_at
--	--

Step 7 — Preference Resolution

Do not simply check: if (userPreference == true) Use a policy abstraction. <pre>public interface NotificationPolicy { PolicyDecision evaluate(NotificationContext context); }</pre>	Possible policies: UserPreferencePolicy QuietHoursPolicy DoNotDisturbPolicy ConsentPolicy FrequencyPolicy TenantPolicy ChannelPolicy	Notification v Policy Engine +---- Userreference +---- Quiet Hours +---- Consent +---- FrequencyLimit +---- Tenant Rules v Allowed Channels
--	---	--

TemplateRendering

Step 8 — Template

id
template_code
event_type
channel
language
subject
body
version
status
created_at
Updated_at

Example:

Template_code: ORDER_CONFIRMED
Channel: EMAIL
Language: en-IN

Step 9 — TemplateResolver

```
interface TemplateResolver {  
    NotificationTemplate resolve(  
        String templateId,  
        ChannelType channel,  
        Locale locale  
    );  
}
```

This allows:

```
ORDER_CONFIRMED  
|  
+---- EMAIL / en-IN  
+---- EMAIL / hi-IN  
+---- PUSH / en-IN  
+---- SMS / en-IN
```

Template Rendering Strategy:

```
public interface TemplateRenderer {  
    RenderedMessage render(  
        NotificationTemplate template,  
        Map<String, Object> data  
    );  
}
```

Implement:

EmailTemplateRenderer
SmsTemplateRenderer
PushTemplateRenderer
WhatsAppTemplateRenderer

Recommended Design Patterns

Strategy Pattern

Provider selection
Retry strategy
Routing strategy
Template rendering
Rate limiting

Factory / Abstract Factory

Provider creation
Channel creation

Chain of Responsibility

Notification policies
Validation
Filtering

Step 10 — Other Considerations

10.1. Priority Example: OTP -> CRITICAL Payment Failed -> HIGH Order Confirmed -> NORMAL Marketing -> LOW Kafka/RabbitMQ can use separate queues/topics:	10.2. Worker Push Workers SMS Workers Email Workers 10.3. Retry Strategy Transient Error (eg: Timeout, Connection reset, HTTP 429, Provider temporarily unavailable, HTTP 503) Permanent Error (eg: Invalid email, Invalid phone number, Invalid template, User blocked, Invalid token)
10.4. Exponential Backoff 10.5. Dead Letter Queue 10.6. Circuit Breaker 10.7. Rate Limiting 10.8. Scheduled Notifications POST /api/v1/notifications/schedule { "userId": "U123", "templateId": "REMINDER", "scheduledAt": "2026-10-01T09:00:00Z" }	10.9. Broadcast / Campaign Architecture 10.10. Multi-Tenant Architecture

Complete Class Diagram

